# Room - Tcpdump: The Basics

- Networking protocols operate behind the scenes, so we normally **don't see actual communication** like ARP requests or TCP handshakes.
- To truly understand how networks work, we need to **capture and analyze network traffic**.
- **Tcpdump** is a command-line tool used for capturing and inspecting packets.
- It is built using the **libpcap** library, which is fast, stable, and widely used in networking tools.
- Originally developed for Unix systems, it was later adapted to Windows as **WinPcap**.

---

## 🎯 Learning Objectives

- Capture network packets and save them to files
- Apply filters to capture specific traffic
- Control how packet data is displayed

Tcpdump normally resolves IP addresses into domain names and port numbers into service names to make the output easier to read. However, this requires additional DNS lookups, which can slow down packet capture and may introduce extra network traffic.

To avoid this, the `-n` option is used to stop the resolution of IP addresses, so only numeric IPs are displayed. The `-nn` option goes further by disabling both IP address resolution and port number resolution, so port numbers like 80 are shown as numbers instead of names like HTTP.

In the given example, Tcpdump was run with the `-n` option along with a packet count limit, capturing and displaying five packets without resolving IP addresses. As a result, the output shows only raw numeric IP addresses and port numbers, making it faster, cleaner, and more suitable for accurate analysis.

| Command | Explanation |
| --- | --- |
| `tcpdump -i INTERFACE` | Captures packets on a specific network interface |
| `tcpdump -w FILE` | Writes captured packets to a file |
| `tcpdump -r FILE` | Reads captured packets from a file |
| `tcpdump -c COUNT` | Captures a specific number of packets |
| `tcpdump -n` | Don't resolve IP addresses |
| `tcpdump -nn` | Don't resolve IP addresses and don't resolve protocol numbers |
| `tcpdump -v` | Verbose display; verbosity can be increased with `-vv` and `-vvv` |

Tcpdump allows filtering based on header bytes using `proto[offset:size]`, giving precise control over packet inspection. For TCP, `tcp[tcpflags]` is used to check specific flags like SYN, ACK, FIN, and RST. By applying logical operations, you can capture packets with exact flag combinations, such as only SYN packets or packets containing SYN or ACK.

You can use `tcp[tcpflags]` to refer to the TCP flags field. The following TCP flags are available to compare with:

- `tcp-syn` TCP SYN (Synchronize)
- `tcp-ack` TCP ACK (Acknowledge)
- `tcp-fin` TCP FIN (Finish)
- `tcp-rst` TCP RST (Reset)
- `tcp-push` TCP Push

Based on the above, we can write:

- `tcpdump "tcp[tcpflags] == tcp-syn"` to capture TCP packets with **only** the SYN (Synchronize) flag set, while all the other flags are unset.
- `tcpdump "tcp[tcpflags] & tcp-syn != 0"` to capture TCP packets with **at least** the SYN (Synchronize) flag set.
- `tcpdump "tcp[tcpflags] & (tcp-syn|tcp-ack) != 0"` to capture TCP packets with **at least** the SYN (Synchronize) **or** ACK (Acknowledge) flags set.

You can write your own filter depending on what you are looking for.

The table below provides a summary of the command line options that we covered.

| Command | Explanation |
| --- | --- |
| `tcpdump -q` | Quick and quite: brief packet information |
| `tcpdump -e` | Include MAC addresses |
| `tcpdump -A` | Print packets as ASCII encoding |
| `tcpdump -xx` | Display packets in hexadecimal format |
| `tcpdump -X` | Show packets in both hexadecimal and ASCII formats |